

```

import numpy as np
import matplotlib.pyplot as plt

c1_c0 = 0.5
pho_c1 = 500
pho_c0 = 340
c = 340
omega = 10
h = 0.1

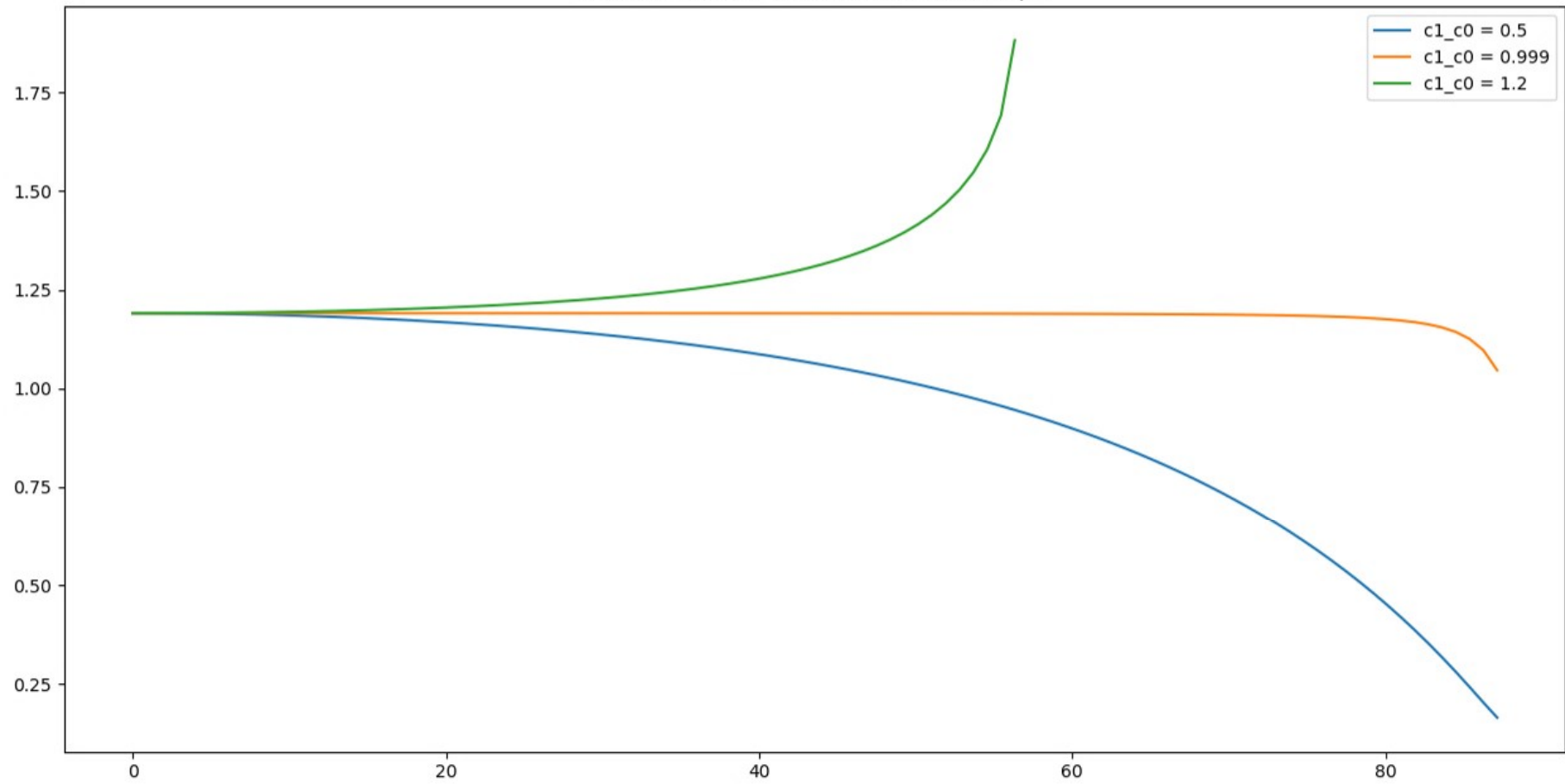
def fact(phi):
    R = np.cos(phi)*pho_c1 - pho_c0*np.sqrt(1.0 - (c1_c0 * np.sin(phi))**2 )
    R = R/ ( np.cos(phi)*pho_c1 + pho_c0*np.sqrt(1.0 - (c1_c0 * np.sin(phi))**2))
    R = R*(np.cos(h*omega /np.cos(phi) / c) -1j*np.sin(h*omega /np.cos(phi) / c) )
    #print(np.cos(phi)[-1])
    return np.abs(1.0+R)

phi = np.linspace(0.0,np.pi/2-0.05,100)

plt.figure(1)
plt.title('Absolute value of Enhancement factor vs phi')
plt.plot(phi*180/np.pi,fact(phi),label = 'c1_c0 = '+str(c1_c0))
c1_c0 = .999
plt.plot(phi*180/np.pi,fact(phi),label = 'c1_c0 = '+str(c1_c0))
c1_c0 = 1.2
plt.plot(phi*180/np.pi,fact(phi),label = 'c1_c0 = '+str(c1_c0))
plt.legend(loc = 'best')
plt.show()

```

Absolute value of Enhancement factor vs phi



```

# Question 3
import numpy as np
import matplotlib.pyplot as plt
from mpldatacursor import datacursor

# Function to calculate dot product
def dot_vec(a_vec,b_vec):
    return np.sum(a_vec*b_vec)

# Function to calculate norm
def norm_vec(a_vec):
    return np.sqrt(np.sum(a_vec**2))

# Function to calculate Re
def Re_cal(x_vect,c,M0,t_array):
    # Mach and velocity vector
    M0_vec = np.array([M0, 0.0 , 0.0])
    V0 = M0_vec*c
    # initializing Re array
    sz = np.shape(t_array)[0]
    Re = np.zeros(sz)
    Re_1 = np.zeros(sz)
    # Different condition for different Mach
    if M0 == 1:
        for i in range(st_ind,sz):
            x_vt = x_vect - V0*t_array[i]
            Re[i] = -0.5*norm_vec(x_vt)**2 / dot_vec(x_vt,M0_vec)
        return Re

    if M0 < 1:
        for i in range(st_ind,sz):
            x_vt = x_vect - V0*t_array[i]
            x_vt_M = dot_vec(x_vt,M0_vec)
            Re[i] = (x_vt_M + np.sqrt( x_vt_M**2 + (1 - M0**2)*norm_vec(x_vt)**2 ) ) /
(1.0 - M0 **2)
        return Re

    if M0 > 1:
        for i in range(st_ind,sz):
            x_vt = x_vect - V0*t_array[i]
            x_vt_M = dot_vec(x_vt,M0_vec)
            discr = x_vt_M**2 + (1 - M0**2)*norm_vec(x_vt)**2
            if discr>0.0:
                # Calculating both the Re
                Re[i] = (x_vt_M + np.sqrt(discr) ) / (1 - M0 **2)
                Re_1[i] = (x_vt_M - np.sqrt(discr)) / (1 - M0 **2)
            # Condition for outside of Mach cone
            if discr<=0.0:
                #print(1)
                pass
        return Re,Re_1

def moving_pt_src(x_vect,c,rho0,M0,A,Omega,t_array):
    # Mach and velocity vector
    M0_vec = np.array([M0, 0.0 , 0.0])
    V0 = M0_vec*c
    # Velocity magnitude
    v0 = M0*c

```

```
# Initializing p_arrays (Two are there for supersonic)
```

```
sz = np.shape(t_array)[0]
p_array = np.zeros(sz, dtype = np.complex)
p_array_1 = np.zeros(sz, dtype = np.complex)
```

```
costh_a = np.zeros(sz)
costh_a_1 = np.zeros(sz)
```

```
# condition for Mach <=1
```

```
if M0 <= 1 :
```

```
    Re = Re_cal(x_vect, c, M0, t_array)
```

```
    for i in range(st_ind, sz):
```

```
        # calculation of te, costh, q, dq_dt, p
```

```
        te = t_array[i] - Re[i]/c
```

```
        costh = dot_vec(V0, x_vect - V0*te)/(norm_vec(V0)*Re[i])
```

```
        q = A*np.exp(Omega*(t_array[i] - Re[i]/c)*1j)
```

```
        q_dot = 1j*Omega*q
```

```
        p_array[i] = q_dot/(Re[i]*(1 - M0*costh)**2)
```

```
        p_array[i] += q*v0*(costh - M0)/( Re[i]**2 * (1 - M0*costh)**3 )
```

```
        p_array[i] *= rho0/(4*np.pi)
```

```
        costh_a[i] = costh
```

```
    return np.real(p_array), costh_a
```

```
# Same explantion follows for M>1 and for both its Re
```

```
if M0 > 1 :
```

```
    Re, Re_1 = Re_cal(x_vect, c, M0, t_array)
```

```
    for i in range(st_ind, sz):
```

```
        if Re[i] != 0.0 :
```

```
            te = t_array[i] - Re[i]/c
```

```
            costh = dot_vec(V0, x_vect - V0*te)/(norm_vec(V0)*Re[i])
```

```
            q = A*np.exp(Omega*(t_array[i] - Re[i]/c)*1j)
```

```
            q_dot = 1j*Omega*q
```

```
            p_array[i] = q_dot/(Re[i]*(1 - M0*costh)**2)
```

```
            p_array[i] += q*v0*(costh - M0)/( Re[i]**2 * (1 - M0*costh)**3 )
```

```
            p_array[i] *= rho0/(4*np.pi)
```

```
            te_1 = t_array[i] - Re_1[i]/c
```

```
            costh_1 = dot_vec(V0, x_vect - V0*te)/(norm_vec(V0)*Re_1[i])
```

```
            q_1 = A*np.exp(Omega*(t_array[i] - Re_1[i]/c)*1j)
```

```
            q_dot_1 = 1j*Omega*q_1
```

```
            p_array_1[i] = q_dot_1/(Re_1[i]*(1 - M0*costh_1)**2)
```

```
            p_array_1[i] += q_1*v0*(costh_1 - M0)/( Re_1[i]**2 * (1 - M0*costh_1)**3 )
```

```
)
```

```
        p_array_1[i] *= rho0/(4*np.pi)
```

```
        costh_a[i] = costh
```

```
        costh_a_1[i] = costh_1
```

```
    if Re[i] == 0.0:
```

```
        p_array[i] = 0.0
```

```
        p_array_1[i] = 0.0
```

```
    return np.real(p_array), np.real(p_array_1), costh_a, costh_a_1
```

```
# Given conditions
```

```
x_vect = np.array([30, 10, 10])
```

```
c = 340
```

```
rho0 = 1.0
```

```
A = 1
```

```
Omega = 100.0
```

```
t_array = np.linspace(0.0, 1.0, 1000)
```

```

# time index at which the first pulse hit the observer so Re makes sense
# it is rounded to 3 decimal because of 1000 points discretization
st_ind = np.where(np.round(t_array,3) == np.round(norm_vec(x_vect)/c,3))[0][0]

M0 = 0.2
plt.figure(1)
if M0<=1:
    p_array,costh = moving_pt_src(x_vect,c,rho0,M0,A,Omega,t_array)
    Re = Re_cal(x_vect,c,M0,t_array)
    #plt.plot(t_array,p_array,'b')
    #plt.plot(t_array,Re,'b')
    #plt.plot(t_array,costh,'b')
    plt.grid('on')

if M0>1:
    p_array,p_array_1,costh,costh_1 = moving_pt_src(x_vect,c,rho0,M0,A,Omega,t_array)
    plt.plot(t_array,p_array,'b')
    plt.plot(t_array,p_array_1,'r')
    #Re,Re_1 = Re_cal(x_vect,c,M0,t_array)
    #plt.plot(t_array,Re,'b')
    #plt.plot(t_array,Re_1,'r')
    #plt.plot(t_array,costh,'b')
    #plt.plot(t_array,costh_1,'r')
    plt.grid('on')

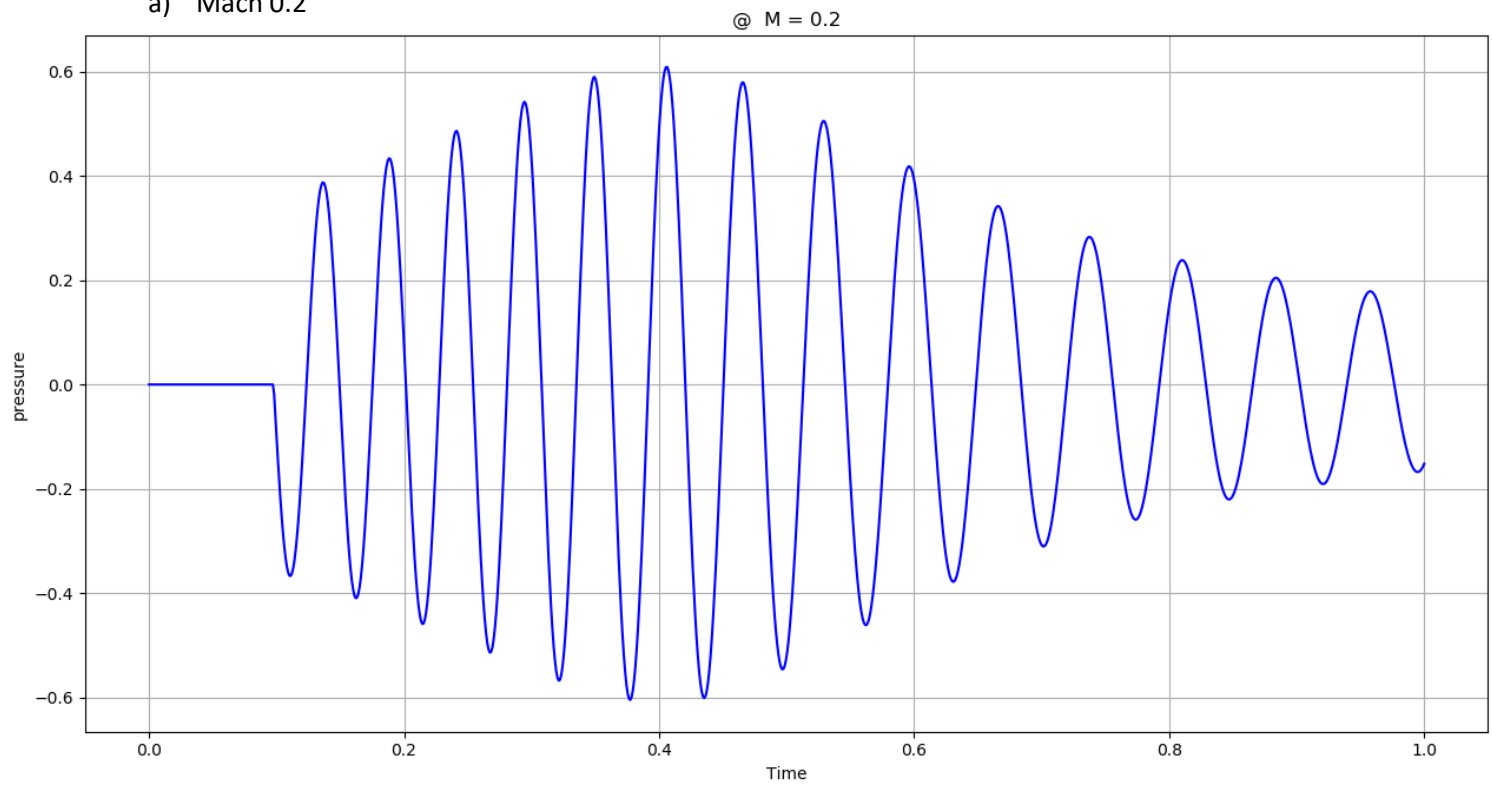
plt.xlabel('Time')
plt.ylabel('pressure')
plt.title('@ M = '+str(M0))

datacursor()
plt.show()

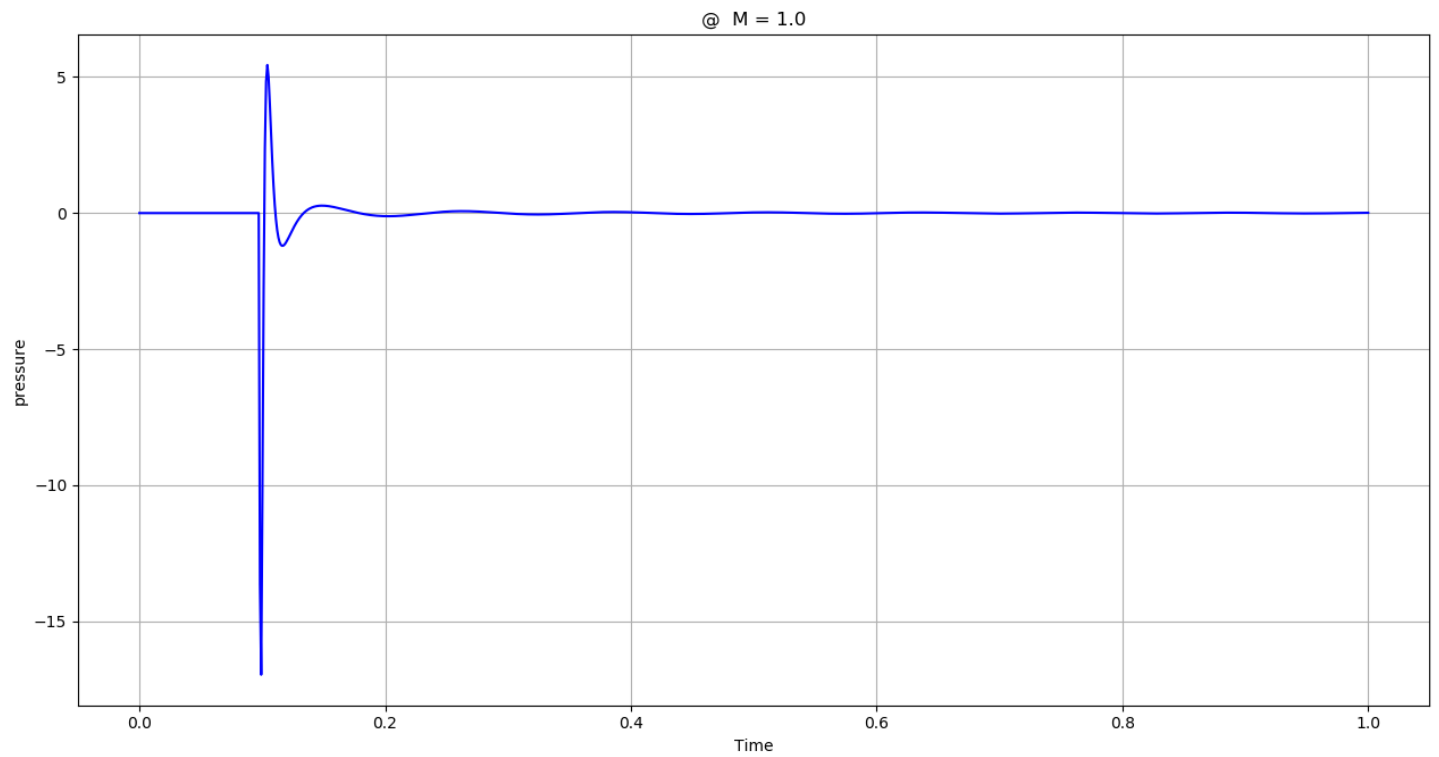
```

Question 4

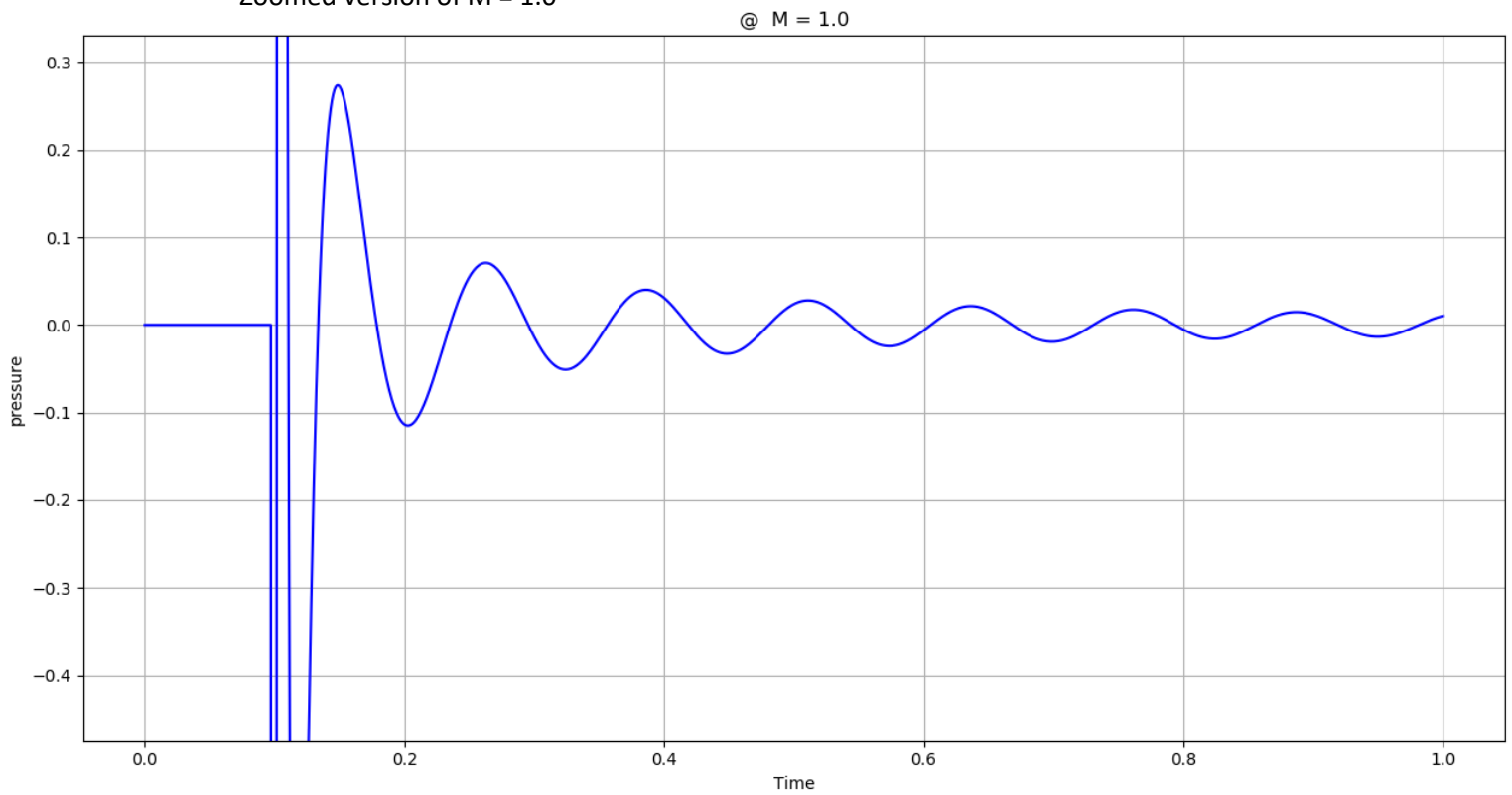
a) Mach 0.2



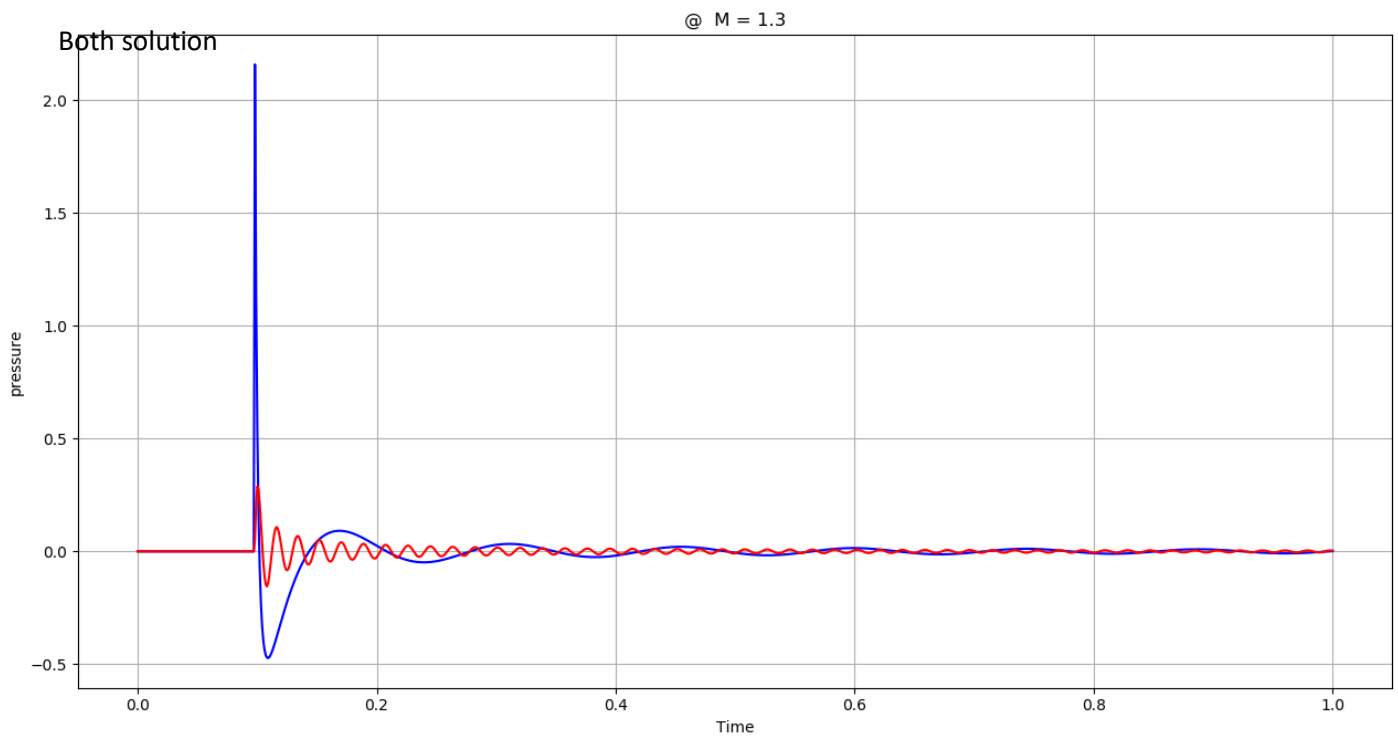
b) Mach 1.0



Zoomed version of M = 1.0

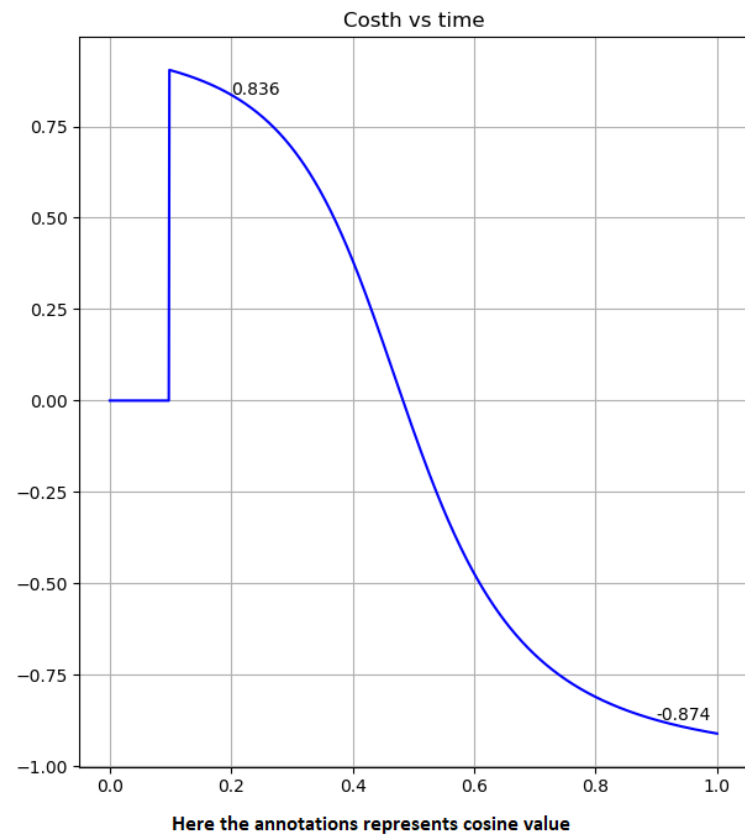
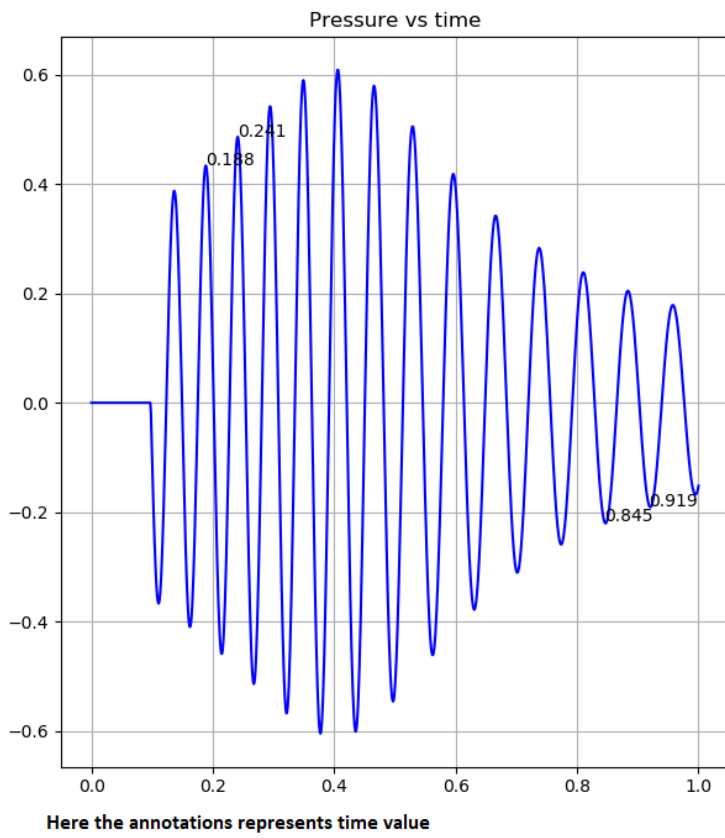


c) M = 1.3



Doppler shift

@ $M = 0.2$



```
# Question no 5
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Radius and angle of the observers
```

```
RObs = 100.0
AngObs = np.arange(20,95.0,5.0)
```

```
# Directory path , note that in the end use \\ instead of \
dirname = 'D:\Study\Sem 8\Assignments\Aeroacoustics\HA5\\'
```

```
# Here I have defined my fourier function
```

```
# trapezoidal rule integration
```

```
# Fourier Function
```

```
def fourier(F,kx,x_src):
    dx_src = x_src[1] - x_src[0]
    N = np.shape(kx)[0]
    four_f = np.zeros(N,dtype = np.complex)
    for i in range(N):
        intgl = F*np.exp(-1j*kx[i]*x_src)
        four_f[i] = np.sum(0.5*(intgl[1:]+intgl[:-1])*dx_src)
    return four_f,kx
```

```
# inverse Fourier function
```

```
def in_fourier(F,X,kx):
    d_kx = kx[1] - kx[0]
    intgl = F*np.exp(1j*kx*X)
    in_four = np.sum(0.5*(intgl[1:]+intgl[:-1])*d_kx) /2 / np.pi
    return in_four
```

```
# hankel Function
```

```
def hankel_2(y,m):
    t1 = -1j*( y - np.pi*( m/2.0 + 1/4.0) )
    t2 = np.sqrt( 2 / (np.pi*y) )
    return t2*np.exp(t1)
```

```
# Function for calcualtion of Xi_r
```

```
def eta_r(D,St_src,Ma,kx):
    temp_1 = (((2*np.pi*St_src*Ma)**2 - (kx)**2) )
    return 1/D *np.sqrt(temp_1)
```

```
def KSAcstcPrArc(dirname,RObs,AngObs):
```

```
    # opening the parameters file
```

```
    param = open(dirname+'parameters.txt','r')
    param = param.read().split('\n')
    params = []
```

```
    for i in range(len(param)-1):
```

```
        params = (param[i].split(' '))
```

```
        # This next step create Ma,St_src,m_src,r_src
```

```
        globals()[params[0]] = np.double(params[2])
```

```
    # Diameter of jet
```

```
    D = 2*r_src
```

```
    # Here I have extracted the data for X
```

```
    x_grid = open(dirname+'x_src.txt','r')
```

```
    x_grid = x_grid.read().split('\n')
```

```

x_src = []
for i in range(len(x_grid)-1):
    x_src.append(np.double(x_grid[i]))
x_src = np.array(x_src)

# Here I have extracted the data for P_real
p_real = open(dirname+'psurf_xs_real.txt','r')
p_real = p_real.read().split('\n')
psurf_real = []
for i in range(len(p_real)-1):
    psurf_real.append(np.double(p_real[i].split(' ')))
psurf_real = np.array(psurf_real)

# Here I have extracted the data for P_imag
p_imag = open(dirname+'psurf_xs_imag.txt','r')
p_imag = p_imag.read().split('\n')
print(p_imag[-1])
psurf_imag = []
for i in range(len(p_imag)-1):
    psurf_imag.append(np.double(p_imag[i].split(' ')))
psurf_imag = np.array(psurf_imag)

# Pressure at the Kirchhoff Surface
p_surf = psurf_real + 1j* psurf_imag

# Radius in cylindrical Coordinates
r = RObs * np.sin(np.pi/180 * AngObs)
x = RObs * np.cos(np.pi/180 * AngObs)

# defining omega
omega_c = 2*np.pi*St_src*Ma/D

# Determining ranges for Kx
kx = D*np.linspace(-omega_c+0.001,omega_c-0.001,1000)
# N.B kx is already normalized

# Intitalizing arrays
four_p_surf = np.zeros(shape = (np.shape(kx)[0],np.shape(p_surf)[1]),dtype =
np.complex)
freq_p_surf = np.zeros(shape = (np.shape(kx)[0],np.shape(p_surf)[1]))
t_sets = np.shape(p_surf)[1]

# calculating Fourier transform
for i in range(t_sets):
    four_p_surf[:,i] , freq_p_surf[:,i] = fourier(p_surf[:,i],kx,x_src)

# PSD and inverse fourier array
f_p_r = np.zeros(shape=(np.shape(r)[0],np.shape(four_p_surf)[1]),dtype=np.complex)
PSD = np.zeros(shape=(np.shape(r)[0]))

# To calculate inverse fourier at the desired X for all blocks
for i in range(np.shape(r)[0]):
    r_temp = r[i]*D
    rks = 3.0*D
    X = x[i]
    for j in range(t_sets):
        kx_j = freq_p_surf[:,j]
        eta_r_temp = r_temp*eta_r(D,St_src,Ma,kx)

```

```

        eta_r_ks = rks*eta_r(D,St_src,Ma,kx)
        f_p_r[i,j] = in_fourier(four_p_surf[:,j]* hankel_2(eta_r_temp,0.0) /
hankel_2(eta_r_ks,0.0) , X,kx)

```

```

# Take the absolute value average along the time blocks

```

```

PSD = np.mean(np.abs(f_p_r)**2,axis = 1)

```

```

return PSD

```

```

# Function call

```

```

PSD = KSAcstcPrArc(dirname,RObs,AngObs)

```

```

# Plotting of data

```

```

plt.figure(1)

```

```

plt.yscale('log')

```

```

plt.xlim(20,90)

```

```

plt.plot(AngObs,PSD,'b',marker='o',fillstyle = 'none')

```

```

plt.xlabel('Observer polar angle, phi , deg')

```

```

plt.ylabel('PSD')

```

```

plt.grid('on')

```

```

plt.show()

```

